HLD Level 1.5

Databases & Storage - C++ / Systems Interview Notes

> **Goal: choose a storage model from the data and access patterns. Do not start with a database brand. Start with: what data exists, how it is read/written, what correctness is required, and how much data/traffic exists.**

1. Start with the access pattern

Before choosing SQL, key-value, document or time-series storage, identify the dominant operations. Is the system read-heavy or write-heavy? Are queries by exact key, range, relationship, timestamp, or arbitrary attributes? Are transactions required? How long is data retained?

```
Data model + access pattern + correctness + scale -> storage choice
```

> **Interview rule: 'Use SQL' or 'use NoSQL' is not an architecture argument. Explain why the workload fits.**

2. SQL / relational databases

Relational databases organize data into structured tables with rows, columns, keys and relationships. They are strong choices when the schema is structured, relationships matter, transactions matter, and the application needs expressive querying.

```
Users(id, name, email)
Orders(id, user_id, total, status)

Users 1 ---- * Orders
```

Typical strengths: structured schema, constraints, joins, transactions, mature query capabilities and strong correctness tools.

3. NoSQL - what the term really means

NoSQL is not one database type. It includes several models optimized for different workloads. Important HLD categories include key-value, document, wide-column and time-series systems.

The useful interview question is not 'SQL or NoSQL?' in isolation. Ask what query patterns, schema flexibility, consistency, transaction scope and scale characteristics are needed.

4. Key-value storage

```
key -> value

device:123 -> { configuration ... }
session:ABC -> { userId, expiry ... }
```

Key-value systems are attractive when access is primarily by a known key. They can be simple and highly scalable, but arbitrary relational queries are not their natural strength.

5. Document storage

Document databases store self-contained records, commonly JSON-like structures. They fit data that naturally forms aggregate documents and may benefit from flexible/evolving fields.

```
device: {
id: 123,
model: camera,
config: {...},
capabilities: [...]
}
```

Flexible schema does not mean 'no data design.' You still need to understand indexes, document size, update patterns and query paths.

6. Time-series storage

Time-series data is organized around timestamps and is common for telemetry, monitoring, metrics, sensor readings and financial market data.

```
(device_id, timestamp, temperature)
(42, 10:00:01, 27.1)
(42, 10:00:02, 27.3)
(42, 10:00:03, 27.2)
```

Typical queries include recent values, time ranges, aggregation over windows and retention/downsampling. This makes time-series-oriented storage a natural consideration for telemetry systems.

7. Object/blob storage

Large binary objects such as images, videos, backups and large files generally should not automatically be placed directly inside a relational row. Object/blob storage is often more appropriate for large immutable/semi-immutable binary content.

```
Metadata DB:
video_id -> owner, timestamp, object_key

Object storage:
object_key -> large video bytes
```

A common architecture stores metadata/queryable fields in a database and the large object itself in object storage.

8. Database indexes

An index is an auxiliary data structure that helps locate records without scanning the entire dataset.

```
Without useful index:
scan many rows -> find user@email

With index on email:
index lookup -> target row(s)
```

Indexes improve selected read/query paths but are not free: they consume storage and must be maintained during writes. More indexes can therefore increase write cost.

> **Interview habit: identify your major queries, then discuss indexes supporting those queries.**

9. Primary keys

A primary key uniquely identifies a record. Key choice affects lookup patterns and, in distributed systems, can later affect partitioning/sharding.

```
user_id -> user record
order_id -> order record
device_id -> device record
```

10. Transactions

A transaction groups operations that should behave as one logical unit. A classic example is transferring money: debit one account and credit another without leaving a partially applied result.

```
BEGIN
debit A
credit B
COMMIT
```

Not every operation needs a multi-record transaction. Strong transactional requirements can constrain storage choices and scaling strategies.

11. ACID

Atomicity: transaction applies completely or not at all.
Consistency: transaction preserves defined database invariants/constraints.
Isolation: concurrent transactions do not produce prohibited interference.
Durability: committed data survives failures according to the database's guarantees.

> **Do not confuse ACID 'consistency' with the distributed-systems strong-vs-eventual consistency discussion. They are related to correctness but refer to different concepts.**

12. Read-heavy vs write-heavy systems

A read-heavy product catalog may benefit from indexes, replicas and caching. A high-volume telemetry ingestion system may care more about sequential/batched writes, partitioning by time/device, retention and write amplification.

```
Read-heavy: reads >>> writes
Write-heavy: writes >>> reads
```

Architecture follows workload. Optimizing every read path can hurt writes, and optimizing only ingestion can make queries unusable.

13. Read/write ratio

Estimate approximate reads/sec and writes/sec separately. 100k total requests/sec means little if you do not know whether it is 99k reads + 1k writes or 1k reads + 99k writes.

14. Query-driven schema design

Design storage around actual queries. For telemetry, examples might be: latest state for one device, all readings for one device in a time range, fleet-level aggregation for the last hour, and alerts by device/time.

Different query patterns may justify different indexes, denormalized views or even different storage systems.

15. Normalization

Relational normalization reduces unnecessary duplication and update anomalies by separating logically distinct entities.

```
User table
Order table -> user_id
Product table
OrderItems -> order_id, product_id
```

The trade-off is that retrieving a complete view may require joins.

16. Denormalization

Denormalization intentionally duplicates or pre-computes data to improve read performance or simplify distributed access.

```
Order view stores:
order_id, user_name, product_name, total ...
```

Reads may become simpler/faster, but updates become harder because multiple copies must remain sufficiently synchronized.

> **Normalization vs denormalization is a trade-off between write/update simplicity, consistency and read efficiency.**

17. Database connection pooling

Opening a fresh database connection for every request is expensive. Services commonly maintain a bounded pool of reusable connections.

```
Worker threads -> Connection Pool -> Database
[c1 c2 c3 ... cN]
```

For C++ services, connection-pool size should be bounded. Too few connections can bottleneck throughput; too many can overload the database. Waiting requests need timeout/backpressure behavior.

18. Prepared statements and batching

Prepared/parameterized statements can reduce repeated parsing/planning overhead and improve safety. Batching multiple writes can improve throughput by amortizing round trips and storage overhead, at the cost of buffering and potentially additional latency.

19. Durability and acknowledgements

Ask what 'write succeeded' means. Was data only accepted into process memory, written to an OS buffer, persisted to durable storage, or replicated? Stronger acknowledgement semantics can increase latency.

```
Client -> Service -> DB memory/buffer -> durable log/storage -> replica(s)
^ where do we ACK?
```

This matters for order systems, payments, telemetry and any workflow where acknowledged data loss has business impact.

20. Retention and lifecycle

Not all data should remain forever at full fidelity. Telemetry/video systems often define hot, warm and archival retention policies.

```
Recent data -> fast/hot storage
Older data -> cheaper storage
Very old -> archive/delete
```

Retention is both a cost and architecture requirement. Consider deletion, compaction, downsampling and legal/business retention rules when relevant.

21. Hot vs cold data

Hot data is frequently/recently accessed and may justify expensive low-latency storage. Cold historical data can often move to cheaper storage with slower retrieval.

22. C++ storage considerations

C++ interviewers may zoom into connection pooling, serialization cost, copying large result sets, buffer ownership, batching, asynchronous I/O, bounded queues, retry semantics and transaction lifetime.

```
Network read -> deserialize -> object allocation -> processing

Questions:
Can copies be reduced?
Can buffers be reused?
Is batching useful?
Who owns result memory?
```

23. Example - telemetry system

For high-volume telemetry, a reasonable design discussion may separate: latest device state, historical time-series data, large images/video, and device/account metadata.

```
Device metadata -> relational/document store
Latest state -> key-value/cache
Telemetry -> time-series-oriented store
Video/images -> object storage
```

One system does not have to use one storage technology for every data type. But every additional datastore increases operational complexity, so justify it.

24. Example - order system

Orders, balances and execution state often have strong correctness and transactional requirements. This pushes the discussion toward controlled authoritative state and careful transaction/ordering semantics rather than blindly optimizing for maximum write throughput.

25. SQL vs NoSQL decision checklist

Ask:

1. What is the data model?
2. What are the primary queries?
3. Do relationships/joins matter?
4. Do multi-record transactions matter?
5. Read/write ratio?
6. Data volume and growth?
7. Schema flexibility?
8. Consistency/correctness requirements?
9. Retention?
10. Partitioning/scaling needs?

26. Common mistakes

SQL cannot scale: false as a blanket statement.

NoSQL always scales better: meaningless without workload/context.

NoSQL means no schema: false; data still has structure and access patterns.

Index everything: indexes cost storage and write work.

Put every binary object in DB rows: often poor for large media workloads.

Transactions are free: stronger coordination/correctness has cost.

One datastore for everything: sometimes fine, sometimes forces poor fits; avoid unnecessary polyglot complexity too.

Ignore retention: storage grows forever.

27. Practice scenarios

1. You are designing a bank-transfer service. Would transactional relational storage be attractive? Why?

2. 1 million cameras send timestamped health metrics continuously. What storage model naturally fits historical queries?

3. Cameras upload 50 MB video clips. Would you store all video bytes directly in normal relational rows? What alternative architecture would you consider?

4. A table has 500 million users and the application frequently queries by email. What concept should you immediately consider?

5. Your system is 99% reads and 1% writes. What optimizations might become attractive?

6. Why can adding many indexes hurt a write-heavy system?

7. A C++ API server creates a new DB connection for every request at 20k RPS. What problem do you see and what would you introduce?

28. 60-second revision

SQL: structured relations, transactions, joins, constraints.
Key-value: direct key-based access.
Document: aggregate JSON-like records/flexible fields.
Time-series: timestamp-oriented telemetry/metrics.
Object storage: large binary objects/files.
Index: faster selected reads at storage/write cost.
Transaction: group operations as a logical unit.
ACID: atomicity, consistency, isolation, durability.
Normalization: reduce duplication.
Denormalization: duplicate/precompute for reads with consistency cost.
Connection pool: bounded reusable DB connections.
Always start with: data + access patterns + correctness + scale.

29. Ready for Level 1.6 when...

You can choose a storage category from access patterns, explain SQL vs major NoSQL models, reason about indexes and transactions, distinguish metadata from large-object storage, and discuss read/write patterns, connection pools and retention.